

用C语言编写简单的BP神经网络

晁飞

May 24, 2012

本文介绍了如何一步步的用C语言编写简单的神经网络。这个神经网络具有一个输入层，一个隐藏层和一个输出层。这个网络可以通过已有的输入向量和输出向量来训练，在训练结束后能通过输入的向量得到输出向量。本文将分为两个部分：第一个是普通BP网络的建立；第二个是BP算法的实现。

BP网络的建立

单一的神经元能够接收输入信号 In ，并得出相应的输出 Out 。比较常见的激活函数是Sigmoid函数：

$$Out = \frac{1.0}{1.0 + \exp^{-In}} \quad (1)$$

尽管其他函数诸如线性或双曲正切函数也被使用，但是Sigmoid函数可以将趋于无限的输入 In 压缩在0到1之间，并且Sigmoid函数的导数形式非常简单：

$$Sigmoid_Derivative = Sigmoid \times (1.0 - Sigmoid) \quad (2)$$

这样简单的形式为用编程语言实现BP的训练算法创造了方便的条件。通常，一个神经元的输入就是其他神经元的输出被加了权值的和值。这就很容易想象到神经网络的传递过程很像是激活信号从各层通过。如果在输入层有 $NumInput$ 个神经元的话，那么一个隐含层的神经元就是所有 $Input[i] \times Weight[i]$ 的和 $SumH$ 。这里 $Weight[i]$ 是输入层中第 i 个输入单元与隐藏层的权值。每个神经元会有一个偏置，被加入到输入之和中，这里偏置被定为 $Weight[0]$ 。它的作用就是神经元的阈值。我们可以通过如下代码来计算隐藏层的值。

```
SumH = Weight[0] ; /*从隐藏的偏置开始*/
for( i = 1 ; i <= NumInput ; i++ ) {
    /*用i循环遍历输入单元*/
    SumH += Input[i] * Weight[i] ;
    /*计算输入与权值的乘积*/
}
Hidden = 1.0 / (1.0 + exp(-SumH)) ;
/*计算通过Sigmoid函数后的值*/
```

由于隐藏层拥有许多神经元，因此将 i 个输入单元与 j 隐藏层神经元之间的权值表示为一个二维的数组 $WeightIH[i][j]$ 。这里 IH 表示的是从输入层到隐藏层。这样第 j 个的隐藏层神经元可以表示为：

```

SumH[j] = WeightIH[0][j] ;
for( i = 1 ; i <= NumInput ; i++ ) {
    SumH[j] += Input[i] * WeightIH[i][j] ;
}
Hidden[j] = 1.0/(1.0 + exp(-SumH[j])) ;

```

有在C语言中，数组的起始位是0，而不是1，所以我们初始化变量时要按如下进行：

```

double Input[NumInput+1] ;
double SumH[NumHidden+1] ;
double Hidden[NumHidden+1] ;
double WeightIH[NumInput+1][NumHidden+1] ;
etc.

```

因为是二维数组，我还要另外一个循环得到各个隐藏层的神经元值：

```

for( j = 1 ; j <= NumHidden ; j++ ) {
    SumH[j] = WeightIH[0][j] ;
    for( i = 1 ; i <= NumInput ; i++ ) {
        SumH[j] += Input[i] * WeightIH[i][j] ;
    }
    Hidden[j] = 1.0/(1.0 + exp(-SumH[j])) ;
}

```

一个隐藏层可以满足大部分的计算需要，因此隐藏层神经元的输出值就可以传递到输出层了。为保持代码的可读性，分别用*i, j, k*来表示每一层的单元个数。并用*WeightIH*和*WeightHO*表示输入层与隐藏层的连接权值，和隐藏层到输出层的连接权值；*SumH*表示输入层到隐藏层输入量，*SumO*表示输入到输出层的输入量；*Hidden*表示通过隐藏层神经元的输出量，*Output*是输出层的输出量。这样整个神经网络的工作过程可以表示为：

```

for( j = 1 ; j <= NumHidden ; j++ ) {
    /* j循环计算隐藏层的输出值 */
    SumH[j] = WeightIH[0][j] ;
    for( i = 1 ; i <= NumInput ; i++ ) {
        /* i循环计算输入层到隐藏层的输入量 */
        SumH[j] += Input[i] * WeightIH[i][j] ;
    }
    Hidden[j] = 1.0/(1.0 + exp(-SumH[j])) ;
}

for( k = 1 ; k <= NumOutput ; k++ ) {
    /* k循环计算输出层的输出值 */
    SumO[k] = WeightHO[0][k] ;
    for( j = 1 ; j <= NumHidden ; j++ ) {
        /* j循环计算隐藏层到输出层的输入量 */
        SumO[k] += Hidden[j] * WeightHO[j][k] ;
    }
    Output[k] = 1.0/(1.0 + exp(-SumO[k])) ;
}

```

对应到BP网络的结构参见例1可以很好的理解上面这段代码。

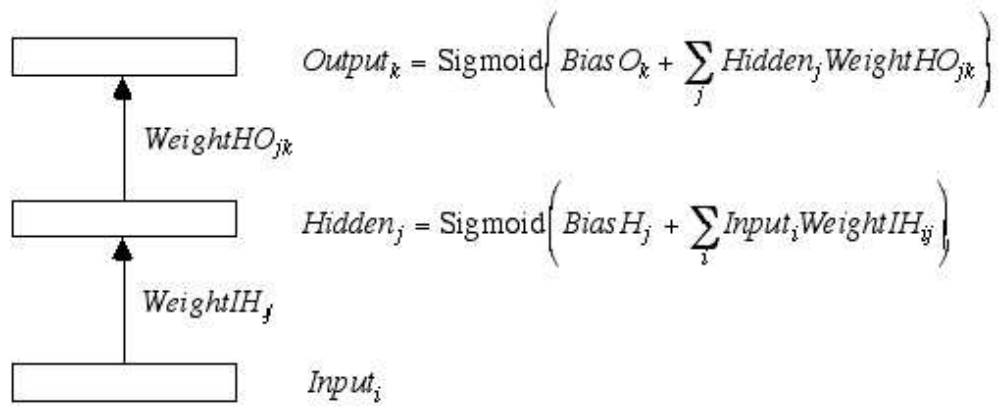


Figure 1: BP网络结构

BP 法的实现

如果我们要进行网络的训练，我们需要一些训练的样本，比如输入与目标输出的向量副：

$\{Input[p][i], Target[p][k]\}$

p 表示有多少个训练样本。网络的训练过程就是将网络输出与目标输出的差值减少到最小。如果使用平方误差来标示所有训练样本的输出单元的误差，那么程序可以表示为：

```
Error = 0.0 ;
for( p = 1 ; p <= NumPattern ; p++ ) {
    for( k = 1 ; k <= NumOutput ; k++ ) {
        Error += 0.5 * (Target[p][k] - Output[p][k])
                  * (Target[p][k] - Output[p][k]) ;
    }
}
```

如果将这段加入到上一节的网络代码中，就可以得出一次训练时整个网络的误差：

```
Error = 0.0 ;
for( p = 1 ; p <= NumPattern ; p++ ) {
    /* p循环遍历所有训练样本 */
    for( j = 1 ; j <= NumHidden ; j++ ) {
        SumH[p][j] = WeightIH[0][j] ;
        for( i = 1 ; i <= NumInput ; i++ ) {
            SumH[p][j] += Input[p][i] * WeightIH[i][j] ;
        }
        Hidden[p][j] = 1.0/(1.0 + exp(-SumH[p][j])) ;
    }

    for( k = 1 ; k <= NumOutput ; k++ ) {
        SumO[p][k] = WeightHO[0][k] ;
        for( j = 1 ; j <= NumHidden ; j++ ) {
            SumO[p][k] += Hidden[p][j] * WeightHO[j][k] ;
        }
        Output[p][k] = 1.0/(1.0 + exp(-SumO[p][k])) ;
        Error += 0.5 * (Target[p][k] - Output[p][k])
                  * (Target[p][k] - Output[p][k]) ;
    }
}
```

下面的工作就是要通过网络的误差来调整网络的连接权值。这里使用了“带冲量的反向传播算法”来训练网络。具体的算法请参考你们前一次的课程实践报告。在程序中用 $\Delta WeightIH$ 和 $\Delta WeightHO$ 来表示权值的修改量：

```
Error = 0.0 ;
for( p = 1 ; p <= NumPattern ; p++ ) {
    /* p循环遍历所有训练样本 */
```

```

for( j = 1 ; j <= NumHidden ; j++ ) {
/* 计算隐藏层的输出值 */
    SumH[p][j] = WeightIH[0][j] ;
    for( i = 1 ; i <= NumInput ; i++ ) {
        SumH[p][j] += Input[p][i] * WeightIH[i][j] ;
    }
    Hidden[p][j] = 1.0/(1.0 + exp(-SumH[p][j])) ;
}

for( k = 1 ; k <= NumOutput ; k++ ) {
/* 计算输出层的输出值和误差 */
    SumO[p][k] = WeightHO[0][k] ;
    for( j = 1 ; j <= NumHidden ; j++ ) {
        SumO[p][k] += Hidden[p][j] * WeightHO[j][k] ;
    }
    Output[p][k] = 1.0/(1.0 + exp(-SumO[p][k])) ;
    Error += 0.5 * (Target[p][k] - Output[p][k])
                * (Target[p][k] - Output[p][k]) ;
    DeltaO[k] = (Target[p][k] - Output[p][k])
                * Output[p][k] * (1.0 - Output[p][k]) ;
}

for( j = 1 ; j <= NumHidden ; j++ ) {
/* 反向传递误差到隐藏层 */
    SumDOW[j] = 0.0 ;
    for( k = 1 ; k <= NumOutput ; k++ ) {
        SumDOW[j] += WeightHO[j][k] * DeltaO[k] ;
    }
    DeltaH[j] = SumDOW[j] * Hidden[p][j] * (1.0 - Hidden[p][j]) ;
}

for( j = 1 ; j <= NumHidden ; j++ ) {
/* 更新权值WeightIH */
    DeltaWeightIH[0][j] = eta * DeltaH[j]
                        + alpha * DeltaWeightIH[0][j] ;
    WeightIH[0][j] += DeltaWeightIH[0][j] ;
    for( i = 1 ; i <= NumInput ; i++ ) {
        DeltaWeightIH[i][j] = eta * Input[p][i] * DeltaH[j]
                        + alpha * DeltaWeightIH[i][j] ;
        WeightIH[i][j] += DeltaWeightIH[i][j] ;
    }
}

for( k = 1 ; k <= NumOutput ; k++ ) {
/* 更新权值WeightHO */
    DeltaWeightHO[0][k] = eta * DeltaO[k]

```

```

        + alpha * DeltaWeightHO[0][k] ;
WeightHO[0][k] += DeltaWeightHO[0][k] ;
for( j = 1 ; j <= NumHidden ; j++ ) {
    DeltaWeightHO[j][k] = eta * Hidden[p][j] * DeltaO[k]
        + alpha * DeltaWeightHO[j][k] ;
    WeightHO[j][k] += DeltaWeightHO[j][k] ;
}
}
}

```

这里 $\Delta WeightIH$ 和 $\Delta WeightHO$ 由两部分来修改：首先， η 是梯度下降参数；其次， α 是BP算法中的冲量参数，它可以有效的保持权值的梯度下降保持一个浮动平均值。 η 与 α 的最优值一般要通过不断试验才能得出。当然 α 应该设为0到1之间，且非零的值可以加速学习速度。但是设为0的话就没有冲量存在了，虽然可以让代码很简单。

η 的值一般依赖于具体的要处理的问题，并与 α 的值也有关系。如果 η 设置的太小，训练的速度将极度缓慢；如果设置的太大，将会导致网络部收敛。通常，将 η 设置为0.1，然后根据结果再双倍的加大或者减半。

最终的训练是将上述的权值改变过程重复多个循环，直到输出误差低于期望值。为了加入这些训练循环，之前的代码可以改成：

```

for( epoch = 1 ; epoch < LARGENUMBER ; epoch++ ) {
/* 带入上面的代码*/if( Error < SMALLNUMBER ) break ;

```

通常还有一些初始化工作要完成，如果将所有权值的初始值都设为0并不能加快网络的训练，因此，所有的权值一般都给一个随机值。在C语言中我们用 $rand()$ 给一个从0到1的随机数，我们一般还用一个小数 $smallwt$ 给出一个权值的参数来控制随机权值的范围，那么代码为：

```

for( j = 1 ; j <= NumHidden ; j++ ) {
/* 初始化WeightIH 和DeltaWeightIH */
    for( i = 0 ; i <= NumInput ; i++ ) {
        DeltaWeightIH[i][j] = 0.0 ;
        WeightIH[i][j] = 2.0 * ( rand() - 0.5 ) * smallwt ;
    }
}

for( k = 1 ; k <= NumOutput ; k++ ) {
/* 初始化WeightHO 和DeltaWeightHO */
    for( j = 0 ; j <= NumHidden ; j++ ) {
        DeltaWeightHO[j][k] = 0.0 ;
        WeightHO[j][k] = 2.0 * ( rand() - 0.5 ) * smallwt ;
    }
}
}

```

一般在初始化时将 $\Delta Weights$ 设为0。而 $smallwt$ 要根据具体问题进行设定，但不能设的太大以防止sigmoid在过早的情况下就饱和了。

如果每个训练循环都以相同的样本训练顺序，会使整个网络产生震荡，因此没一次的训练，我们都采取一个随机的循序进行训练。相关的代码为：

```

for( p = 1 ; p <= NumPattern ; p++ ) {
/* 设置顺序数组 */
    ranpat[p] = p ;
}
for( p = 1 ; p <= NumPattern ; p++ ) {
/* 随机的交换位置 */
    np = p + rando() * ( NumPattern + 1 - p ) ;
    op = ranpat[p] ;
    ranpat[p] = ranpat[np] ;
    ranpat[np] = op ;
}

```

到此为止，一个简单的BP网络就实现了，源代码中用BP网络模拟了XOR的功能，希望本文能帮助你们理解那些代码。